School of Information and Computer Technology
Sirindhorn International Institute of Technology, Thammasat University
CSS326 Database Programming Laboratory

Laboratory Assignment #9: Transactions

Objectives:

- To run multi-step operations as transactions with COMMIT and ROLLBACK.
- To make a route atomic in mysql2 using a single pooled connection.
- To roll back on broken business rules (such as insufficient stock) and prove atomicity.

Scenario: in your bookstore app, placing an order should record the order **and** reduce the book's stock. Cancelling should delete the order **and** restore the stock. These multi-step actions must be all-or-nothing, so you will wrap them in transactions, exactly as you did for enrol and un-enrol in the lab exercises.

Setup

Download and unzip **Lab9_Assignment_Starter.zip**. In MySQL Workbench, run `sql/bookstore_setup.sql` once. It creates a separate `bookstore_lab9` database, so your earlier bookstore data is left untouched:

```
books(     id, title, author, price, stock )
customers( id, name, email )
orders(    id, customer_id, book_id, quantity )  -- FKs to customers and books

-- a transaction here: reduce books.stock AND insert the order, together
```

Open `db.js` and set `user` and `password` to your own MySQL login (the database name is already `bookstore_lab9`). Then, in a terminal in the project folder:

```
npm install     # once: downloads express and mysql2
npm run dev     # starts the server on http://localhost:3000 (Ctrl + C stops it)
```

The two routes in `controllers/orderController.js` already work, but they are **not atomic**. Rewrite them using the pattern `pool.getConnection()` → `beginTransaction()` → `commit()` / `rollback()` → `release()` in `finally`. Guard the stock with `UPDATE … WHERE id = ? AND stock >= ?` so it only succeeds when enough is available. Ready-made test requests are in `requests.http`, and `GET /books` shows the current stock. This assignment is marked out of **12**.

Questions

1. **In Workbench.** Write a transaction that reduces one book's stock and inserts a matching order, wrapped in START TRANSACTION … COMMIT. Then run the same two statements but ROLLBACK instead, and in one sentence describe what the data looked like afterwards. Use `sql/q1_workbench.sql`. **(1.5 points)**

2. **Transactional order.** Rewrite `POST /customers/:id/orders` (body `{"book_id": 1, "quantity": 2}`) so that, inside a transaction, it decrements the book's stock by the ordered quantity and inserts the order, then commits and returns **201**. Use getConnection, beginTransaction, commit, rollback and release. Before borrowing a connection, reject a `book_id` or `quantity` that is not a positive whole number with **400**. **(2 points)**

3. **Roll back on a rule.** Guard the stock update with `WHERE id = ? AND stock >= ?`. If it affects no rows (not enough stock), roll back and return **409** with a clear message. No order may be created. **(1.5 points)**

4. **Roll back on an error.** Show that if the INSERT of the order fails **after** the stock update, the stock change is undone. For example, place an order as customer 999, who doesn't exist, so the foreign key rejects the INSERT. Describe how you forced the failure and show the book's stock before and after. **(1.5 points)**

5. **Transactional cancel.** Rewrite `DELETE /customers/:id/orders/:orderId` so that, inside a transaction, it reads the order's book and quantity, deletes the order, and adds the quantity back to the book's stock: both, or neither. Return **404** (and roll back) if the order does not exist **for that customer**. **(2 points)**

6. **Prove atomicity.** Record a book's stock, place an order for it, then cancel that order. Include screenshots showing the stock decrease on order and return to its original value after cancel. **(1.5 points)**

7. **Short answer.** Explain atomicity in your own words, and give one real example, outside this lab, where a transaction is essential. **(1 point)**

8. **Short answer.** Explain why a transaction must run on a single connection obtained with `pool.getConnection()`, rather than calling the pool directly. **(1 point)**

Submission

- Zip your project folder **without** the `node_modules` folder, plus one PDF with your screenshots (Q1, Q3, Q4, Q6) and your answers to Q7 and Q8.
- Each screenshot should show the request (or SQL) **and** the result, such as the response and status code, or the stock value.
- Due date: _____